



University
of Glasgow

W6-03: Log-Structured-Merge (LSM) Trees

COMPSCI4064 (H) and COMPSCI5088 (M)

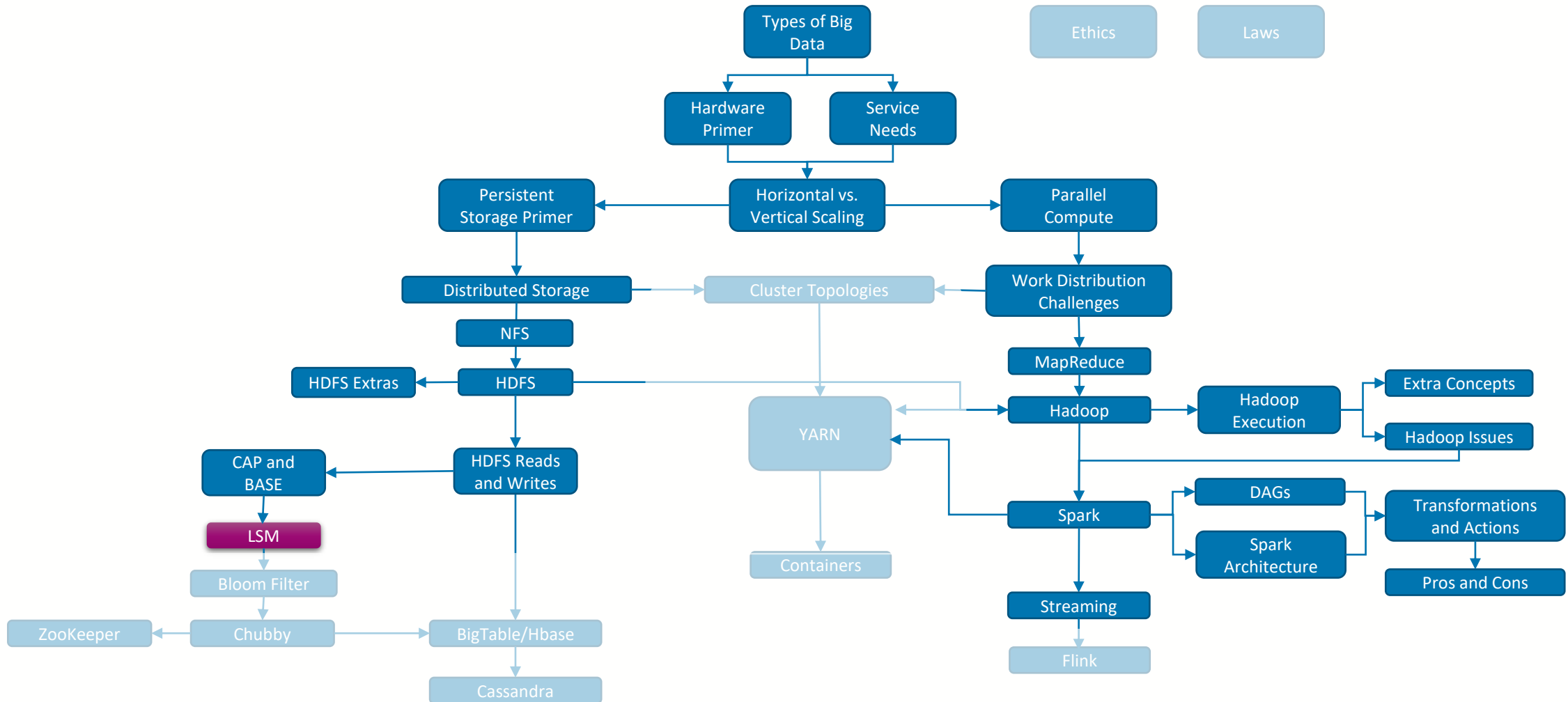
Dr. Richard McCreddie

WORLD
CHANGING
GLASGOW

A WORLD
TOP 100
UNIVERSITY



Where are we?





The Index Problem

- One of the major challenges when we increase the replication within a distributed system is **how we keep track of where all the replicas are stored**
 - This is what the Name Node does in the HDFS, and recall that we noted that its not suited to lots of small files
- Any distributed storage system will need to hold an index that specifies the **physical locations** of whatever storage units we are using (e.g. files or blocks)
 - This index always needs to be **available** and must be **fast to search**



Random Reads/Writes

- In the persistent storage primer we discussed how disk storage (the most common type of persistent storage) is:
 - **Very Slow** for **Random** reads/writes
 - **Fast** for **Sequential** reads/writes (up-to 3 orders of magnitude faster)
- **Most storage index solutions will be random read heavy**, since by their nature they are looking up a record specifying where our data is stored
 - So... store the index in main memory?
 - But this cannot scale to very large indices
 - We also need to persist it regularly so we don't lose knowledge of data
- So we need a way to **read and write index data to disk**
 - Most Big Data solutions store index data as a **Log** of records



Using a Log for Indexing

- Core Concepts:
 - **Write**: Every time a data storage unit is written (e.g. a file or block) write a log entry with a:
 - Data Identifier
 - Timestamp
 - Location
 - (maybe other data)
 - **Read**: When we want to read that data again, we need to find the associated log entry, meaning we need to search the log
 - **Update**: If we need to update or delete data we need to alter the log somehow



Using a Log for Indexing

Advantages

- Solves slow writes by sequentially writing entries to a file
 - No complex structure management
- Can be spread over multiple files (partitions) by key

Disadvantages

- Reads require you to search the log
 - We need to find a way to do this fast
- Log editing (updates and deletes) can be problematic, as we to first find then alter or replace a record



How do we find something in a log?

- Reads are the real issue with log indexes, how might we solve this?
 - Sort the log by key; use binary search
 - Involves an expensive sort and may take time to scan to the desired record
 - Split the log into partitions by hash of key; only search the relevant partition
 - Better, but partitions may be large and need to be managed
 - Use a tree-like structure at the storage level (e.g., B+ tree, ISAM, ...)
 - Even better, but we now need to define up-front a complex tree data structure and manage writes to it, which will be slow
 - Use an externally managed index
 - Not a bad option but runs into scalability issues

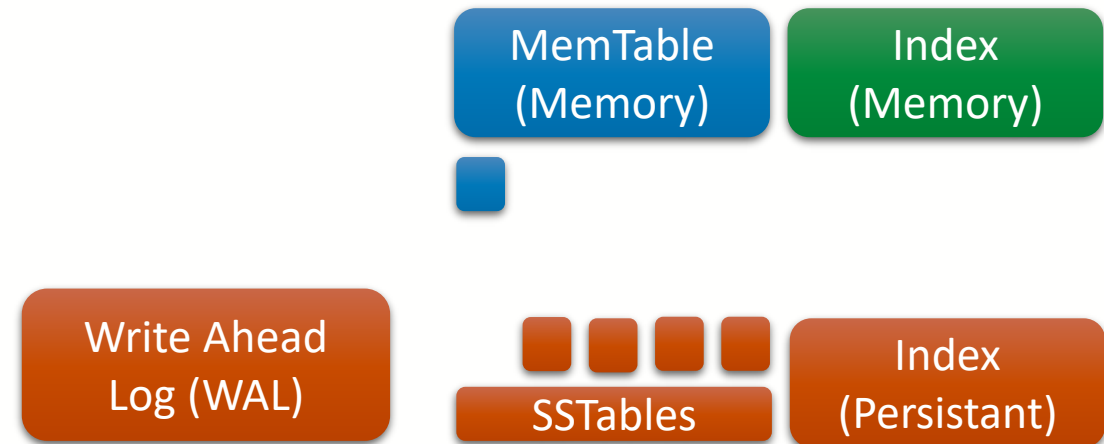


Log-Structured-Merge (LSM) Tree

- A data structure designed to enable fast reading and writing of logs
 - Used by Cassandra (discussed later) to store its storage index
- Core Concepts
 - Write log entries in **batches** to **spill files**
 - **Sort** each batch in memory before writing
 - Spill files are **immutable** (can't be changed after write)
 - Log **updates** get written to **new spill files**
 - Periodically **merge spill files**
 - **Read only** a subset of the **spill files** that are likely to **contain the data we need**
 - Use **Bloom Filters** to identify what spill files to read (more on this later)
 - **Deletes** are written to **Tombstone files**
 - Read Tombstones during spill file merges to avoid persisting 'deleted' data forever

Log-Structured-Merge (LSM) Tree Components

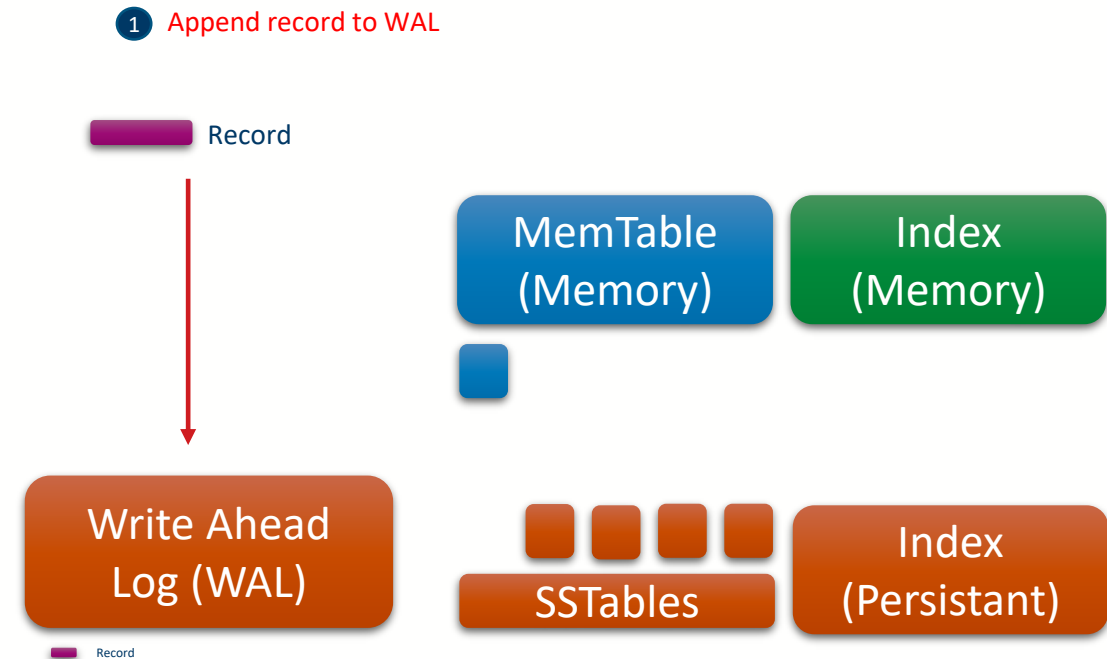
- **Write Ahead Log (WAL):** Persistently stores write records between spill file writes
- **MemTable:** Stores the most recent batch of write records in memory. Records are keyed to the files they are about. Read requests check this first.
- **SSTable:** Comprised of a series of spill files. Once the MemTable becomes full, it gets sorted, converted to a spill file and added to the SSTable.
- **Index:** Contains a mapping between file keys and what (persisted) SSTables contain information about that key
- **Compactor:** Merges SSTables to reduce fragmentation of information about a key





LSM Tree Write

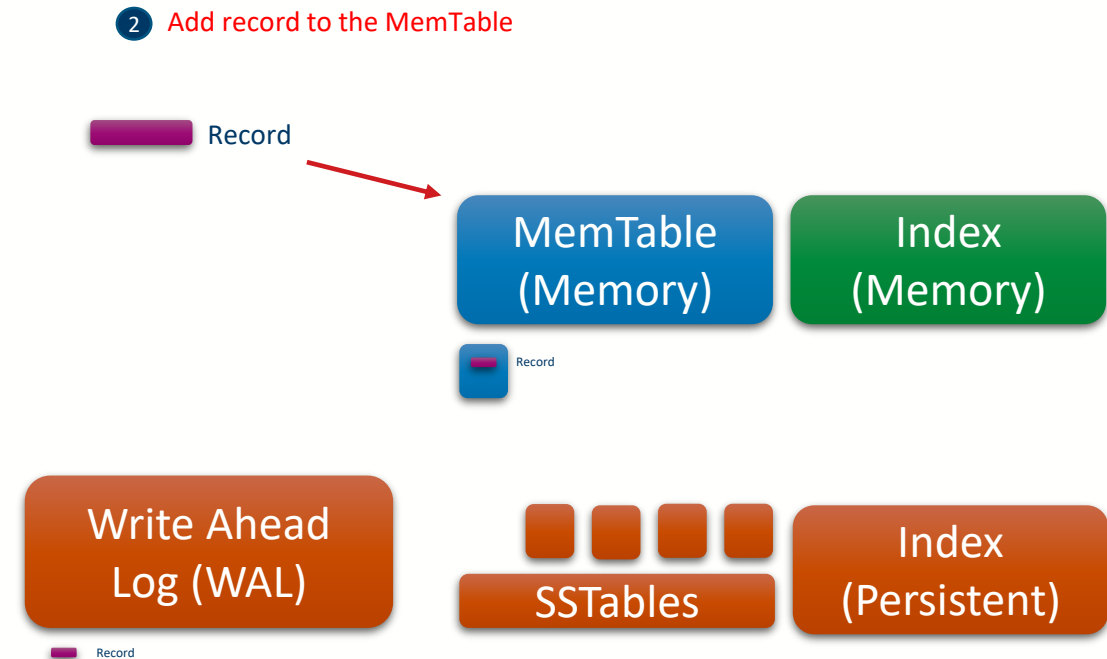
1. Write the record to the WAL





LSM Tree Write

1. Write the record to the WAL
2. Add record to the MemTable

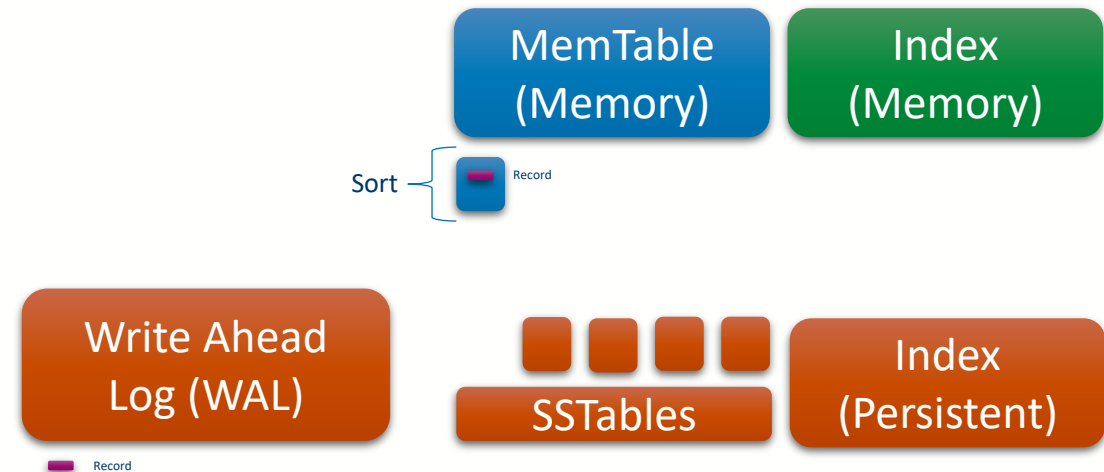




LSM Tree Write

1. Write the record to the WAL
2. Add record to the MemTable
3. If MemTable is full
 1. Sort MemTable

3 Sort MemTable

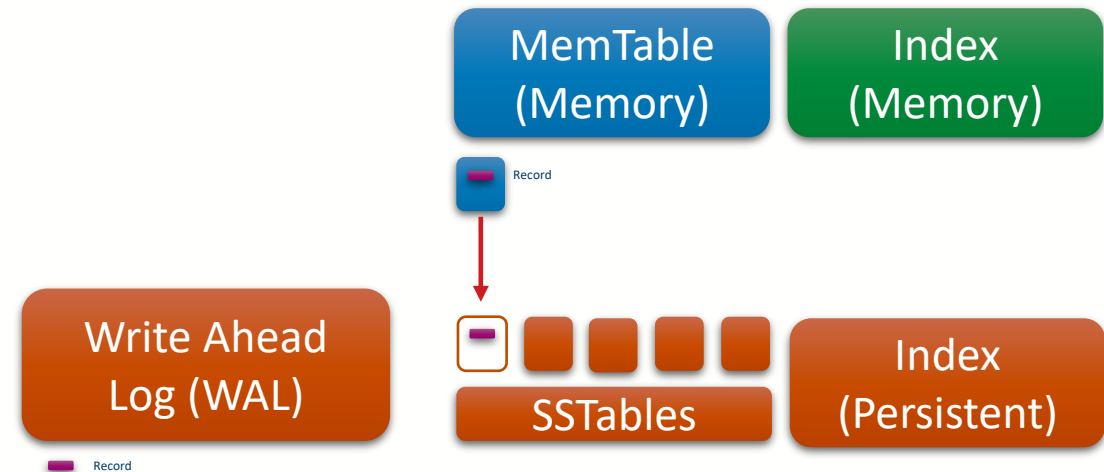




LSM Tree Write

1. Write the record to the WAL
2. Add record to the MemTable
3. If MemTable is full
 1. Sort MemTable
 2. Write MemTable to Spill File

3 Write MemTable to Spill File

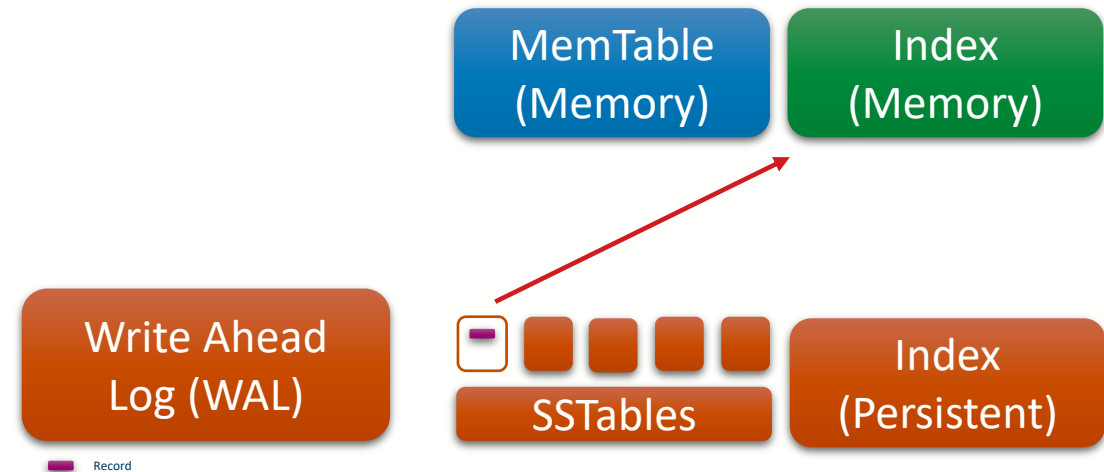




LSM Tree Write

1. Write the record to the WAL
2. Add record to the MemTable
3. If MemTable is full
 1. Sort MemTable
 2. Write MemTable to Spill File
 3. Update Index (with keys for that SSTable)

3 Write Update Index

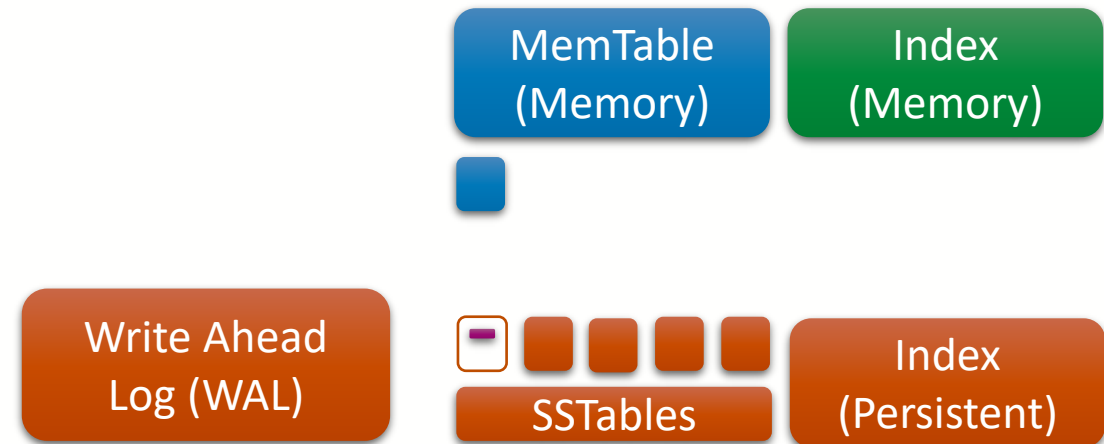




LSM Tree Write

1. Write the record to the WAL
2. Add record to the MemTable
3. If MemTable is full
 1. Sort MemTable
 2. Write MemTable to Spill File
 3. Update Index (with keys for that SSTable)
 4. Clear WAL

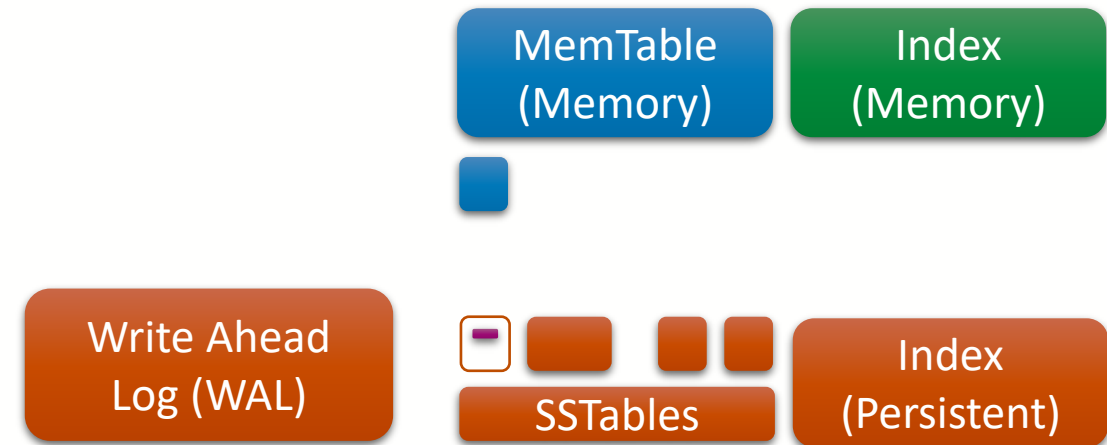
3 Clear WAL





LSM Tree Write

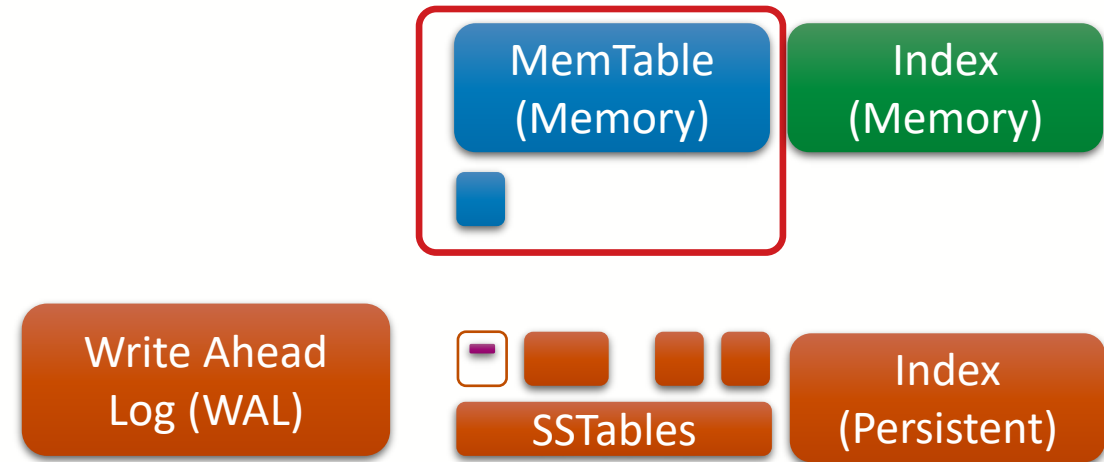
1. Write the record to the WAL
2. Add record to the MemTable
3. If MemTable is full
 1. Sort MemTable
 2. Write MemTable to Spill File
 3. Update Index (with keys for that SSTable)
 4. Clear WAL
4. In Background
 1. Merge SSTables (remove old data from updates and deletes)





LSM Tree Read

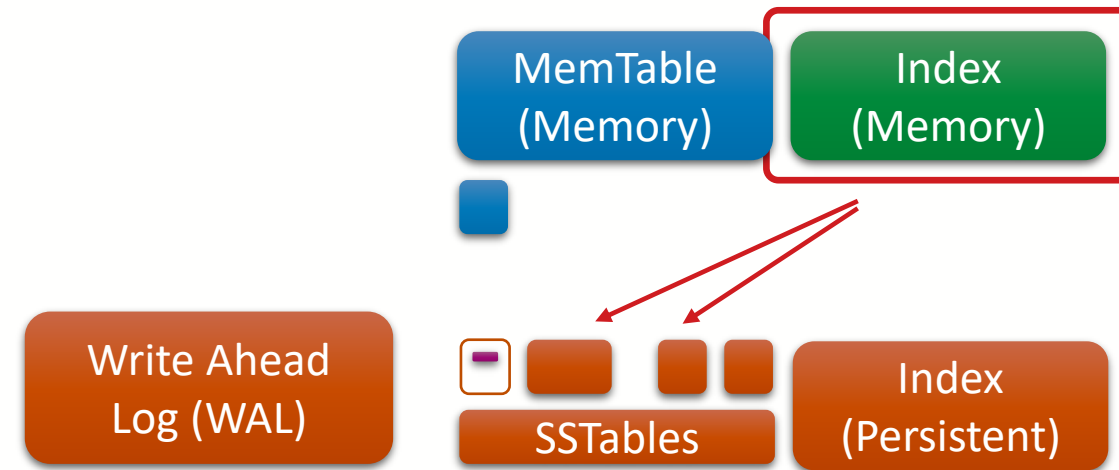
1. Check if the MemTable has the desired record





LSM Tree Read

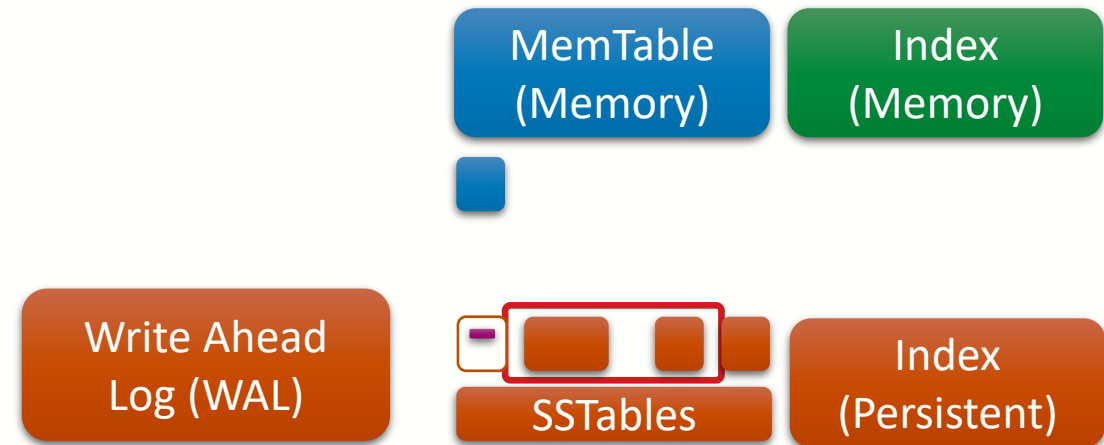
1. Check if the MemTable has the desired record
2. If not, ask the index for the list of SSTables that might contain the record
 1. Uses a Bloom Filter for this





LSM Tree Read

1. Check if the MemTable has the desired record
2. If not, ask the index for the list of SSTables that might contain the record
 1. Uses a Bloom Filter for this
3. Load the SSTables into memory and scan them for the record





Key Design Decisions

- Writes:
 - Uses a Write Ahead Log to make sure we always have a write record persistently stored somewhere if a failure occurs
 - The Write Ahead Log is append only, so relatively fast to write to
 - The log structure is edited in memory first (MemTable), and only persisted as a batch write when needed, cuts down on disk access
- Reads:
 - Assuming we are more likely to need recent data than old data, we should get fast returns via the MemTable (acts like a recent cache)
 - Otherwise, we do a single fast request to the index ($O(1)$ complexity) then need to scan the appropriate SSTable/Spill Files
 - Block reads the entire file (fast sequential reads), then scan in memory
- Updates and Deletes:
 - Not a great solution here, needs to use the Compactor to clean up and remove old redundant data, which is expensive
 - But we can schedule that to run during quiet times



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow